```
»           }
        }
    }
```

Finally, back in `WebFragment`, call `viewModel.registerToken()` in the token change handler. Wrapping this call in a launched `lifecycleScope` ensures it occurs on a background thread.

ch10_65/android/app/src/main/java/com/masilotti/hikingjournal/fragments/WebFragment.kt

```
    package com.masilotti.hikingjournal.fragments

    import android.Manifest
    import android.os.Build
    import androidx.activity.result.contract.ActivityResultContracts
»   import androidx.lifecycle.lifecycleScope
    import com.google.firebase.messaging.FirebaseMessaging
»   import com.masilotti.hikingjournal.viewmodels.NotificationTokenViewModel
    import dev.hotwire.navigation.destinations.HotwireDestinationDeepLink
    import dev.hotwire.navigation.fragments.HotwireWebFragment
»   import kotlinx.coroutines.launch

    @HotwireDestinationDeepLink("hotwire://fragment/web")
    class WebFragment : HotwireWebFragment() {
»       private val viewModel = NotificationTokenViewModel()
        // ...

        private fun registerForTokenChanges() {
            val firebase = FirebaseMessaging.getInstance()
            firebase.token.addOnCompleteListener { task ->
                if (task.isSuccessful) {
»                   viewLifecycleOwner.lifecycleScope.launch {
»                       viewModel.registerToken(task.result)
»                   }
                }
            }
        }
    }
```

Everything is now in place to register an FCM token and receive notifications. Let’s see it in action.